



Programación Orientada a Objetos POO404

Unidad 1



GUIA DE EJERCICIOS #1

1. La siguiente actividad será elaborada **solamente en Grupos de 4 o 5 compañeros**.
2. A continuación, se describe un conjunto de ejercicios a ser resueltos en Java, aplicando los principios de la programación orientada a objetos.
3. Cada ejercicio debe ser resuelto en un proyecto diferente. Cada uno debe ser resuelto de acuerdo con las condiciones especificadas en la descripción de este.
4. Para evitar descuentos generales en la nota de un ejercicio, se debe cumplir cada una de las restricciones brindadas en la descripción correspondiente.
5. Los identificadores por utilizar deben utilizar las reglas de nomenclaturas usadas por las comunidades de desarrolladores de Java.
6. El código fuente elaborado para la solución se debe documentar con un lenguaje técnico “de programador a programador” (por bloques de líneas y líneas específicas, claves en la lógica de la solución) y de manera impersonal.
7. Cada Ejercicio se solucionará en un Proyecto Java distinto y tomar en cuenta los aspectos siguientes:
 - Antes de la definición de su metodo main, incluir como comentario a la lista de TODOS los integrantes del Equipo que entrega la tarea.
 - Elaborar un diagrama UML con los paquetes, clases y asociaciones implementadas en su respectiva solución. El UML debe realizarse con el software Draw.IO. El archivo generado (.drawio) debe exportarse también a un archivo de imagen (.png o .jpg); para luego, guardar ambos archivos en la carpeta de solución del proyecto.
 - Comprimir (en .zip o .rar) a la carpeta del proyecto.
 - Elegir a un compañero del Equipo para que suba el archivo comprimido del proyecto de solución por el Enlace de envío creado por su docente en el aula digital.

Importante:

- Cada ejercicio resuelto sera enviado por un integrante distinto del Equipo.
- **Cualquier código “extraño” en la solución presentada (que implemente temas aun no vistos en clase de teoría) o que haya sospecha de ser resuelto con el uso de IA, hará que El ejercicio sea ignorado.**

- Si estudiantes no están de acuerdo con la pérdida de nota de un Ejercicio (por sospechas brindadas por docente), deberán defender el Ejercicio y explicar perfectamente a TODAS las preguntas que docente considere necesarias para demostrar la autenticidad del código fuente correspondiente.

Ejercicios a Resolver

32% Ejercicio 1: Abstracción de un Conjunto Matematico

Un *Conjunto* es una coleccion de elementos de un mismo tipo, los cuales nunca se repiten en su contenido.

Se necesita diseñar un objeto en Java que implemente el concepto de un Conjunto para almacenar elementos de tipo entero.

Para guardar los elementos del conjunto, el objeto debe utilizar un objeto de la clase `ArrayList`.

El objeto debe tener un método distinto para resolver estas tareas:

- Retornar (con sentencia `return`) a la cantidad de elementos almacenados del conjunto.
- Comprobar si un elemento (recibido en un parametro) pertenece al contenido actual del conjunto. Si ya existe elemento, se retorna `true`.
- Insertar elemento al conjunto, recibido como parametro. Retorna `true` solamente si pudo insertar valor recibido en el contenido de elementos del conjunto.
- Extraer elemento del conjunto. Retorna (con `return`) a un elemento solicitado en un parametro. Si encuentra el valor, lo elimina del contenido y lo retorna. De lo contrario, retorna `null`.
- Retornar en un parametro a un vector con la copia de todos los elementos almacenados. Si el conjunto no tiene elementos, retorna `null` en el parametro.

Restricciones:

Almacenamiento:

Solo puede utilizar `ArrayList` para almacenar a los datos del conjunto. Cualquier otra clase de coleccion utilizada para este fin, ocasionara que no se califique este ejercicio.

Encapsulamiento:

Todos los campos de la clase deben tener visibilidad privada, por lo tanto, desde las instancias de esta clase se podra acceder a ellos solamente por los accesores `getter` y/o `setter`, segun solicite el ejercicio. Cada campo no encapsulado, ocasionara un descuento del 25% en la nota obtenida.

Del objeto ArrayList:

Se permite utilizar solamente a sus métodos `add` y `remove`. El uso de cualquier otro miembro de esta clase `java.util.ArrayList`, ocasionara un descuento del 25% en la nota de este ejercicio.

La clase solicitada no puede utilizar a la clase `Iterator`.

Calidad de la documentacion interna insertada en el cod. fuente

Diagrama UML de clases:

Cada clase y otros elementos son incluidos. Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta.

Todas las clases y paquetes creados son incluidas en el diagrama. Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta y valor inicial (si lo requiere).

Cada método definido pero no invocado sera descartado y se penalizara en la nota de este Ejercicio.

Demostracion:

-Debe implementarse 3 objetos de la clase solicitada.

-Diseñar una demostración para verificar el funcionamiento de cada método solicitado en la descripcion de este Ejercicio.

-Cada método definido en una clase, pero que no sea invocado, será descartado y se penalizara en la nota de este Ejercicio.

33% Ejercicio 3: Representacion de Fracciones

Un numero fraccionario (una fracción) es una división indicada entre 2 valores enteros, denominados numerador y denominador. Ejemplos de estas pueden ser $\frac{4}{6}$, $\frac{-57}{14}$, $\frac{-7}{22}$

Existen 2 tipos de fracciones: *fracciones propias* (ej. $\frac{9}{16}$, $-\frac{2}{5}$, etc.) y *fracciones impropias* (ej. $-\frac{86}{13}$, $\frac{41}{26}$, etc.). Incluso, los números enteros pueden ser representados como fracciones, ej: numero entero 9 puede definirse como la fracción $\frac{9}{1}$

Las fracciones impropias pueden ser representados como *números mixtos*, que se conforman por un numero entero y una fracción impropia, ej.: la fracción $\frac{-57}{14}$ es equivalente al numero mixto $-4\frac{1}{14}$

Y por ultimo, las fracciones pueden ser simplificadas, por ej: la fracción $-\frac{24}{30}$ puede ser simplificarse como $-\frac{4}{5}$

Con la introducción anterior sobre los números fraccionarios, proceda a desarrollar el siguiente programa en Java:

Cree un paquete (diferente al usado para la clase Main y su metodo main de inicio). En este nuevo paquete, elaborar una clase para representar a una fraccion. Debe implementar encapsulamiento (getter, setter) y cuidar que se cumpla que el denominador de la fracción siempre sea positivo (mayor que cero). El valor de la fracción predeterminada de los objetos de esta clase sera $\frac{0}{1}$ el cual representa al valor entero 0. El constructor debe ser sobrecargado para que una 2da definición acepte el par de valores de la fracción en parámetros y una sobrecarga mas para recibir solo un numero entero (y luego ser guardado como una fraccion).

La clase debe implementar un método público que permita simplificar el valor de la fracción almacenada en los campos de la clase.

Y, por último, implementar un accesor público de solo lectura (getter) que retorne (con *return*) a un String con la representación de la fracción almacenada en el objeto. Observe la siguiente tabla, el cual muestra ejemplos de cómo debe ser devuelto el String solicitado:

<i>Fraccion almacenada en objeto</i>	<i>String a devolver</i>
$\frac{4}{7}$	4/7
$-\frac{26}{11}$	-2(4/11)
$-\frac{9}{1}$	-9
$\frac{28}{3}$	9(1/3)

Luego, crear otra clase, la cual implementara a 4 metodos estáticos. El 1er primer metodo estático debe recibir en parámetros a 2 instancias de su clase que representa fracciones y retornar (con *return*) a una nueva instancia con el resultado de la suma. Por ej.

Si el par de instancias a sumar tienen almacenadas a las fracciones $\frac{-4}{9}$ y $\frac{1}{6}$, respectivamente, este metodo para sumar devolverá una fracción con el valor $\frac{-5}{18}$. Otro ejemplo: la suma de $\frac{-1}{6}$ y $\frac{-1}{3}$ genera la fracción $\frac{-1}{2}$

El 2do metodo estático hará la multiplicación de 2 objetos de su clase para almacenar fracciones y devolverá con return a una instancia con el resultado, ej.: el producto de $\frac{3}{7}$ y $\frac{14}{9}$ genera la fracción $\frac{2}{3}$

El 3er y ultimo metodo deben calcular el MCM y MCD, respectivamente. Ambos reciben a 2 numeros enteros y retornan (con return) el valor matemático correspondiente.

Ejemplos:

- El MCM de 4 y 3 es 12.
- El MCD de 3 y 5 es 1.
- El MCM y MCD de 15 y 18 es 90 y 3, respectivamente.

Demostracion:

-no se necesita interactuar con usuario. Simplemente debe demostrarse el funcionamiento de las 2 clases solicitadas en este ejercicio.

-Por lo tanto, para iniciar, debe implementarse 5 objetos de la clase para representar fracciones, que incluyan fracciones propias e impropias, así como fracciones que representen a enteros y fracciones invalidas (como $\frac{6}{0}$ o $\frac{-9}{-4}$).

-Luego, imprimir los valores almacenados en cada objeto, utilizando el getter que devuelve su valor almacenado, en el formato solicitado en la descripción de este ejercicio.

-A continuación, demostrar el uso de los métodos estáticos de la otra clase implementada, utilizando como argumentos a los objetos de fracciones ya existentes en la demostración.

Restricciones:

-Implementar el encapsulamiento solicitado a todos los campos de ambas clases, implementando en los setter a la lógica de negocios descrita en el ejercicio.

-Cada método solicitado en el ejercicio y definido en el código, pero que no sea utilizado/invocado, será descartado y se penalizará en la nota de este Ejercicio.

- Debe crearse el Diagrama UML de clases que incluya a todas las clases, desde la que define al método estático main hasta las 2 clases solicitadas en este ejercicio y las asociaciones correspondientes entre ellas.

- Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta y valor inicial (si lo requiere) creado en el código fuente.

35% Ejercicio 3: Registro de recursos audiovisuales

Una mediateca cuenta con un conjunto de libros, revistas, DVDs y CDs de audio y otros recursos, disponibles para sus socios. Se pide realizar un programa en JAVA que permita gestionar el material disponible por la mediateca. El programa debe tener las siguientes opciones:

- Listar materiales disponibles
- Buscar material
- Agregar material
- Modificar material
- Borrar material
- Salir

Los datos asociados a cada tipo de material son los siguientes:

- Para los libros: código de identificación interna, título, autor, número de páginas, editorial, ISBN, año de publicación y unidades disponibles.

- Para las revistas: código de identificación interna, título, editorial, periodicidad, fecha de publicación y unidades disponibles.

- Para los CDs de audio: código de identificación interna, título, artista, género, duración, número de canciones y unidades disponibles.

- Para los DVDs: código de identificación interna, título, director, duración, género.

El código de identificación interna es una convención utilizada por la mediateca para identificar de forma única a cada material disponible. Dicho código se genera de la siguiente manera.

- Para los libros: palabra "LIB" más número correlativo de 5 dígitos (Ejemplos: LIB00007, LIB66251, LIB55842).

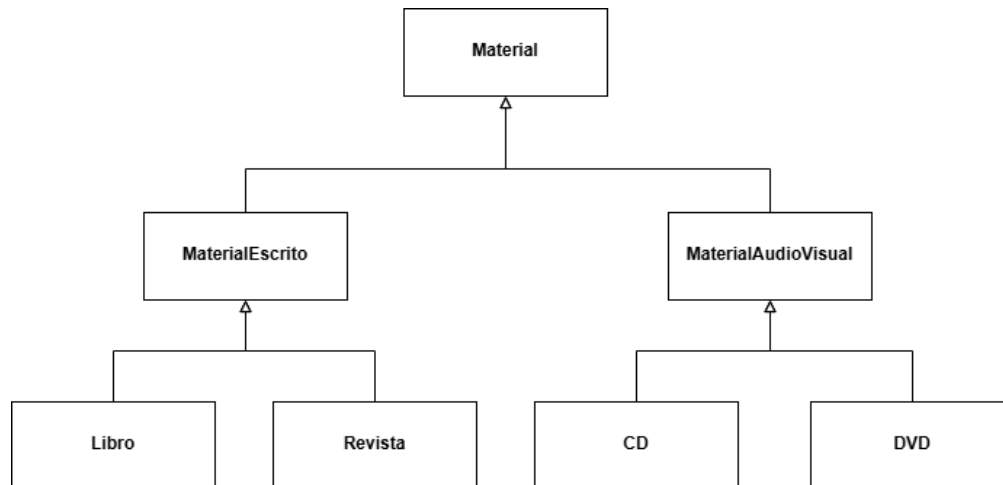
- Para las revistas: palabra "REV" más número correlativo de 5 dígitos (Ejemplos: REV00257, REV62351, REV59823).

- Para los CDs de audio: palabra “CDA” más numero correlativo de 5 dígitos (Ejemplos: CDA00248, CDA12345, CDA55842).

- Para los DVDs: palabra “DVD” más numero correlativo de 5 dígitos (Ejemplos: DVD00008, DVD66252, DVD55843).

Consideraciones:

- Para implementar la selección de las diversas opciones de menus, debe hacer la lectura directa del carácter (sin necesidad de presionar Enter para confirmar), utilizando al metodo System.in.read
- Tenga en cuenta las respectivas validaciones para los datos de entrada deben ser realizados por las clases del modelo de gestión de datos, no desde el metodo main.
- Deberá hacer uso de colecciones ArrayList para almacenar los diversos objetos de recursos audiovisuales gestionados por el programa.
- Deberá utilizar programación orientada a objetos, implementando los conceptos de herencia y clases abstractas de tal forma que se implemente la siguiente jerarquía de clases:



Restricciones:

-Implementar el encapsulamiento apropiado (solo lectura, solo escritura o escritura-lectura, según se justifique) a los diferentes campos de la jerarquía de clases.

-Cada método solicitado en el ejercicio y definido en el codigo, pero que no sea utilizado/invocado, será descartado y se penalizara en la nota de este Ejercicio.

- Debe crearse el Diagrama UML de clases que incluya a todas las clases, desde la que define al metodo estático main hasta el árbol de clases solicitados, actualizados con los miembros implementados en java y las asociaciones correspondientes entre ellas.

- Cada miembro de clase se incluye apropiadamente en el diagrama, con la visibilidad correcta y valor inicial (si lo requiere) creado en el código fuente.

Demostracion:

-Elaborar un menu con las opciones descritas al inicio de este ejercicio.

-Debido a que se crearan objetos de diferentes clases pero todos derivados de una clase base común, se debe crear un único objeto de la clase ArrayList, que almacene el listado de objetos de los diferentes tipos de medios que usuario vaya creando o modificando o los remueva/elimine al usar la app. Investigar uso de palabra instanceof.

-Las verificaciones de datos ingresados por usuario solo deben ser hechas por los setters definidos en el esquema de clases en la descripción de este ejercicio. Es el objeto el que debe indicar que todo esta bien o hay algún tipo de problema y que usuario debe corregir.